PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Curso de Engenharia de Software

Disciplina: Testes de Software

Detecção de Bad Smells e Refatoração Segura

Luidi Cadete Silva

Matrícula: 804945

Novembro de 2026

1. Diagnóstico e análise estática inicial

Objetivo: melhorar a legibilidade e a manutenção de um gerador de relatórios JavaScript, usando testes e análise estática como evidências. Base: commit 39d39f9 do repositório do professor; implementação em arquivo separado.

Bad Smells identificados manualmente

Tipo e localização	Impacto na manutenção e nos testes
Método longo generateReport, linhas 11-69	Reúne seleção, prioridade e apresentação. Dificulta localizar defeitos e compreender os cenários de teste.
Condicionais complexas Laço, linhas 27-56	Perfis, valores e formatos se cruzam em vários níveis. Aumenta a dificuldade de conferir caminhos e limites.
Duplicação de lógica CSV/soma, linhas 35-37 e 46-48	A mesma mudança precisa ser repetida em ramos diferentes, criando risco de divergência e regressão.

Achado adicional: os números 500 e 1000 foram substituídos por constantes com nomes de negócio. Esses limites merecem verificações específicas.

Execução anterior à criação do arquivo refatorado



Figura 1. Captura autêntica da página local que executou npx eslint src/ e exibiu stdout/stderr reais. Logs e metadados estão em docs/evidencias/baseline.

O SonarJS mediu complexidade cognitiva 27 (limite 15) e indicou um if simplificável: 2 erros, 0 avisos. A ferramenta tornou objetiva a dificuldade percebida manualmente, mas não apontou todos os smells: duplicação de trechos e números mágicos permanecem achados manuais.

2. Refatoração e diferença autorizada

O smell mais crítico foi a combinação de método longo e condicionais aninhadas em generateReport. O recorte original abaixo mistura autorização, escolha do formato, renderização e soma. Os trechos são extraídos dos arquivos reais.

Antes - src/ReportGenerator.js, linhas 43-54

```
43        } else if (user.role === 'USER') {
44          // Users comuns só veem itens de valor baixo
45          if (item.value <= 500) {
46            if (reportType === 'CSV') {
47              report += `${item.id},${item.name},${item.value},${user.name}\n`;
48              total += item.value;
49            } else if (reportType === 'HTML') {
50              report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
51              total += item.value;
52            }
53          }
54        }
```

Depois - decisões e responsabilidades separadas

```
1  const STANDARD_USER_VALUE_LIMIT = 500;
2  const ADMIN_PRIORITY_THRESHOLD = 1000;
```

```
14  function canViewItem(user, item) {
15    return user.role === 'ADMIN'
16      || (user.role === 'USER' && item.value <= STANDARD_USER_VALUE_LIMIT);
17  }
```

```
52      for (const item of items) {
53        updatePriority(user, item);
54        if (!canViewItem(user, item)) continue;
55        const row = createRow(reportType, user, item);
56        if (!row) continue;
57        body += row;
58        total += item.value;
59      }
60
```

Extract Method: cabeçalho, linha, rodapé, visibilidade e prioridade ficaram em funções pequenas. Decompose Conditional: canViewItem expressa a regra de acesso e o laço usa continuações simples. A construção de linhas e a acumulação do total deixaram de ser duplicadas entre perfis.

Correção de formatação, além da refatoração

O original gera `<tr >` em linhas de ADMIN sem estilo; o teste exige `<tr>`. No refatorado, o espaço passou a integrar apenas o atributo de estilo. Essa diferença foi autorizada e documentada; não é apresentada como equivalência textual absoluta.

Preservaram-se a classe exportada, o construtor, db, generateReport e as mutações de item.priority, inclusive em formato desconhecido. Os arquivos originais estão intactos; a cópia da suíte altera exclusivamente o import.

3. Validação e conclusão

Execução real	Aprovados	Falhas	Total
Jest original	4	1	5
Jest refatorado	5	0	5
Jest completo	9	1	10

A falha original no HTML de ADMIN ocorreu antes e depois da refatoração. Não houve aprovação total: npm test retorna código 1. Não se alteraram expectativas, não se excluíram testes e não se introduziram mocks para esconder a falha.

Métrica com critérios iguais	Original	Refatorado
Erros de ESLint / avisos	2 / 0	0 / 0
Complexidade cognitiva de generateReport	27	5

Redução de 100% dos erros e de 81,5% da complexidade do método. O maior valor de uma função refatorada é 5. A medição suplementar usa a API com limite zero para revelar valores; o lint oficial manteve limite 15 e as mesmas regras.

A auditoria complementar aprovou 321 comparações exatas de saídas e mutações, além dos limites 500/501 e 1000/1001, prioridade preexistente, itens vazios e entradas de perfil/formato desconhecidos. A diferença HTML foi validada separadamente com saídas completas, sem normalização. Hashes e blobs Git confirmaram a preservação dos originais.

Conclusão

A separação de responsabilidades e a redução dos smells facilitam a leitura e as mudanças futuras. Os testes funcionaram como rede de segurança durante a refatoração e também revelaram a inconsistência preexistente. O lint mede parte da melhoria estrutural, enquanto os testes verificam comportamentos. Suítes aprovadas aumentam a confiança, mas não provam ausência de defeitos em todas as entradas.

Reprodução e referências

Ambiente: Node 24.14.1, npm 11.11.0, Jest 29.7.0, ESLint 8.57.1 e SonarJS 0.21.0. Jest requer a flag ESM; ESLint 8 foi mantido por compatibilidade com .eslintrc.json. Manifestos, JSON e saídas completas: docs/evidencias. Implementação: 2383b83. Assistência de IA (Codex) declarada no README.

Fork: github.com/LuidiC/bad-smells-js-refactoring
Branch: trabalho/refatoracao-segura
Base: github.com/CleitonSilvaT/bad-smells-js-refactoring
Jest 29.7: jestjs.io/docs/29.7/ecmascript-modules
SonarJS: github.com/SonarSource/eslint-plugin-sonarjs/tree/0.21.0
FOWLER, Martin. Refatoração: Aperfeiçoando o Design de Códigos Existentes. 2. ed. Novatec, 2019. Catálogo: refactoring.com/catalog/.